

Disciplina: Qualidade de Software e Testes

Professor: Leonardo Cardia

Diagnóstico Técnico

Gilded Rose — Modernização de Sistema Legado

Discentes:

Andre Junio Deomondes Carvalhal

Gianluca Motta Lopo

Igor Lima Carvalho

Lucca Torres Badaró Silvani

Ryan Luigi Paixão da Luz

2026

Diagnóstico Técnico — Código Legado

Este documento apresenta a análise técnica realizada sobre o método `update_quality`, localizado no arquivo `legacy/gilded_rose.py` (linhas 8 a 36), responsável por toda a regra de atualização diária dos itens do estoque no sistema original.

Problemas identificados

1. Condicionais profundamente aninhados

O trecho localizado entre as linhas 26 e 36 do arquivo `legacy/gilded_rose.py` apresenta até quatro níveis de condicionais aninhados dentro de um único bloco de código. Essa estrutura dificulta a leitura e a compreensão do fluxo lógico, pois para entender o efeito de uma única condição é necessário interpretar todas as condições que a envolvem. Como consequência, qualquer alteração em uma regra exige a releitura cuidadosa do bloco inteiro, aumentando o risco de introdução de erros.

2. Identificação do tipo de item por comparação repetida de texto

O código identifica o tipo de cada item comparando diretamente o seu nome com textos fixos, repetidos em diversos pontos do método: o nome "Aged Brie" aparece três vezes (linhas 10 e 27), o nome "Sulfuras, Hand of Ragnaros" aparece quatro vezes (linhas 12, 24 e 30) e o nome "Backstage passes to a TAFKAL80ETC concert" aparece três vezes (linhas 10, 17 e 28). Essa repetição representa uma duplicação do conhecimento de domínio: caso o nome de um item precise ser alterado no futuro, será necessário atualizar manualmente cada um desses pontos, com risco real de esquecimento.

3. Mistura de responsabilidades em um único método

O método `update_quality` é responsável, ao mesmo tempo, por decidir a direção da variação de qualidade (se ela aumenta ou diminui), a intensidade dessa variação, a regra especial dos itens Backstage Passes e a atualização do número de dias restantes (`sell_in`). Um único método que concentra decisões distintas para cinco tipos diferentes de item viola o princípio de responsabilidade única, tornando o código mais difícil de manter e de testar.

4. Regra específica embutida em um bloco genérico

Nas linhas 17 a 23, a regra particular dos itens Backstage Passes (ganho adicional de qualidade quando a data do show se aproxima) está implementada dentro do mesmo bloco condicional genérico que trata o item Aged Brie. Essa organização cria um acoplamento desnecessário entre duas regras de negócio que não têm relação entre si.

5. Duplicação da regra de limite de qualidade

Os limites de qualidade (mínimo 0 e máximo 50) não estão centralizados em nenhum ponto do código. A verificação `quality < 50` aparece quatro vezes (linhas 15, 19, 22 e 35) e a verificação `quality > 0` aparece de forma equivalente em outros três pontos. Essa duplicação

obriga o desenvolvedor a lembrar de aplicar a mesma regra em todos os lugares onde a qualidade é alterada, o que é propenso a falhas.

6. Baixa testabilidade

Não é possível testar isoladamente a regra de um único tipo de item, como o Aged Brie, por exemplo. Para validar qualquer regra é necessário instanciar a classe `GildedRose` com uma lista completa de itens e executar o método `update_quality` por inteiro. Essa limitação foi confirmada na execução da suíte de testes original, registrada no arquivo `docs/evidencias/testes_legacy_antes.txt`: a cobertura real de regras de negócio é de 0%, pois um dos testes existentes é apenas um exemplo de modelo (não testa nenhuma regra real) e o outro é um teste de ponta a ponta, não unitário.

7. Regra de negócio incompleta — bug confirmado

O cenário de demonstração do sistema, definido em `legacy/texttest_fixture.py`, já inclui um item chamado "Conjured Mana Cake". No entanto, o método `update_quality` não possui nenhuma regra específica para esse tipo de item, o que faz com que ele seja tratado como um item comum: perde 1 ponto de qualidade por dia antes do vencimento e 2 pontos por dia após o vencimento. De acordo com a especificação do problema, um item Conjurado deveria perder qualidade duas vezes mais rápido que um item comum (2 pontos por dia antes do vencimento e 4 pontos por dia após o vencimento). Esse comportamento incorreto foi confirmado na execução registrada em `docs/evidencias/execucao_legacy_antes.txt`, onde se observa que o item "Conjured Mana Cake" perde exatamente 1 ponto de qualidade por dia. Trata-se da lacuna que o Passo 8 do trabalho solicita corrigir.

Por que esses problemas representam um risco

A concentração de toda a lógica em um único método torna qualquer nova regra de negócio, como a dos itens Conjurados, mais difícil de implementar com segurança: é preciso reler e potencialmente alterar um método que já trata quatro tipos diferentes de item, o que aumenta o risco de causar uma regressão em alguma regra que não deveria ter sido alterada.

A ausência de testes unitários por tipo de item obriga a equipe a validar qualquer alteração executando a simulação completa do sistema e inspecionando manualmente a saída, o que não é uma prática escalável nem confiável a longo prazo.

Por fim, a duplicação de nomes de itens e de limites de qualidade cria múltiplos pontos onde uma mesma correção precisa ser replicada. Corrigir uma regra em um desses pontos e esquecer o equivalente em outro é um erro fácil de cometer e difícil de detectar sem uma rede de testes automatizados.

Ressalvas — o que não deve ser considerado problema

É importante não exagerar o diagnóstico. O método analisado é relativamente curto, com cerca de trinta linhas — o problema identificado é de natureza estrutural (acoplamento excessivo,

duplicação de conhecimento e identificação de tipo por comparação de texto), e não está relacionado ao volume de código.

Além disso, apesar de difícil leitura, a lógica do código legado está correta para todos os itens já previstos na especificação original. Essa correção foi confirmada por execução direta do sistema, registrada em docs/evidencias/execucao_legado_antes.txt. A única regra de negócio ausente, de fato, é a do item Conjurado.